

SIMATS ENGINEERING
ASSESSMENT TOOL 2 : Scenario-Based

**CSA1511 – CLOUD COMPUTING AND BIG DATA
ANALYTICS**

Name : NAVADEEP R

Reg No : 192424110

Faculty Incharge : Dr.HEMAVATHI R

CO2 – ASSESSMENT TOOL – 2

Scenario-Based

1. A media company wants to migrate its video streaming platform to the cloud.

The system must handle millions of concurrent users with minimal buffering.

Design a cloud architecture using scalability and caching techniques.

Explain how load balancing and auto-scaling can ensure smooth streaming.

Discuss cost optimization strategies for such high-demand workloads.

Problem Statement

A media company wants to migrate its video streaming platform to the cloud. The platform must support millions of concurrent users while minimizing buffering and maintaining smooth video delivery.

Proposed Cloud Architecture

A scalable architecture can be designed as:

Users



DNS / CDN



Load Balancer



Auto-Scaling Application Servers



Video Storage + Database



Monitoring & Analytics

1. Content Delivery Network (CDN)

A CDN caches video content at geographically distributed edge locations. Users receive videos from a nearby edge server instead of the main cloud server.

Benefits:

- Reduces latency.
- Reduces load on origin servers.
- Improves streaming speed.
- Minimizes buffering.

2. Load Balancing

A load balancer distributes incoming requests across multiple application servers.

For example:

1 Million Requests



Load Balancer



Server 1 | Server 2 | Server 3

This prevents a single server from becoming overloaded.

3. Auto-Scaling

Auto-scaling automatically increases or decreases the number of application servers according to traffic.

High traffic → Add servers

Low traffic → Remove unnecessary servers

This ensures sufficient capacity during peak streaming hours.

4. Scalability Techniques

- Horizontal scaling of application servers.
- CDN-based video caching.
- Object storage for large video files.
- Database replication and caching.
- Auto-scaling based on CPU, requests, or concurrent users.

Cost Optimization

1. Use CDN caching to reduce origin bandwidth costs.
2. Use object storage instead of expensive compute storage.
3. Use auto-scaling to shut down unused resources.
4. Use reserved/committed pricing for predictable workloads.

5. Use appropriate video compression and adaptive bitrate streaming.
6. Store rarely accessed videos in cheaper storage tiers.

Conclusion

A combination of CDN, load balancing, auto-scaling, caching, and cloud object storage can support millions of concurrent users while reducing buffering and controlling operational costs.

2. A fintech startup is deploying applications using containers across multiple cloud providers.

They face challenges in maintaining consistency and managing deployments.

Propose a solution using container orchestration and multi-cloud strategies.

Explain how service discovery and scaling can be handled efficiently.

Highlight the benefits and risks of a multi-cloud deployment model.

Problem Statement

A fintech startup deploys containerized applications across multiple cloud providers but faces problems with deployment consistency, service discovery, and scaling.

Proposed Architecture

Developer Code



Container Image / Registry



Kubernetes Clusters



Cloud Provider A Cloud Provider B

Each cloud can run similar containerized workloads using a common orchestration platform such as **Kubernetes**.

1. Container Orchestration

Kubernetes can manage:

- Container deployment.
- Container health.
- Scaling.
- Service discovery.
- Rolling updates.

- Failure recovery.

Using the same container images and deployment configurations across providers improves consistency.

2. Consistent Deployment

A **CI/CD pipeline** can automatically:

1. Build the application.
2. Create the container image.
3. Run security and testing checks.
4. Push the image to a container registry.
5. Deploy the same image to multiple cloud clusters.

Infrastructure-as-code can also be used to maintain consistent configurations.

3. Service Discovery

Kubernetes provides internal service discovery using services and DNS.

For example:

Payment Service



Kubernetes Service Discovery



Payment Container Instances

Applications can locate services without depending on changing container IP addresses.

4. Scaling

Kubernetes can automatically scale containers based on workload.

Low traffic → Fewer containers

High traffic → More containers

Horizontal Pod Autoscaling can use metrics such as CPU utilization and request load.

Benefits of Multi-Cloud

1. **Reduced vendor dependency** – The organization is not completely dependent on one provider.
2. **Improved availability** – Another cloud can continue serving users if one provider experiences an outage.
3. **Geographic flexibility** – Applications can be deployed closer to users.
4. **Cost flexibility** – Workloads can be distributed according to pricing and resource availability.

Risks of Multi-Cloud

- Increased management complexity.
- Different cloud services and configurations.

- Higher networking costs.
- Difficult monitoring across providers.
- Security and identity management challenges.
- Data synchronization complexity.

Conclusion

Using Kubernetes, container registries, CI/CD, service discovery, and automated scaling provides a consistent multi-cloud container environment. However, multi-cloud increases operational complexity and requires strong security, monitoring, and centralized management.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.

The organization needs a robust backup and disaster recovery strategy.

Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.

Provide steps to ensure business continuity during failures.

Problem Statement

An e-learning platform suffers data loss due to accidental deletion. The organization requires a backup and disaster recovery system to protect student records, courses, assignments, and other important data.

Proposed Architecture

Application



Primary Cloud Database / Storage



Backup + Versioning



Secondary Region / Disaster Recovery Environment

1. Cloud-Native Backup

Regular automated backups should be configured for databases, files, and application configurations.

For example:

Primary Database

↓ Automated Backup

Backup Storage

This allows deleted or corrupted data to be restored.

2. Versioning

Object storage versioning maintains previous versions of files.

If a user accidentally deletes:

Assignment.pdf

the previous version can still be recovered.

3. Replication

Important data should be replicated to a separate availability zone or region.

Primary Region

↓

Replication

↓

Secondary Region

If the primary region becomes unavailable, the secondary environment can be activated.

4. RTO – Recovery Time Objective

RTO represents the maximum acceptable time required to restore the service after a failure.

Example:

RTO = 30 minutes

The organization should design recovery mechanisms that bring the application back within 30 minutes.

5. RPO – Recovery Point Objective

RPO represents the maximum acceptable amount of data loss measured in time.

Example:

RPO = 5 minutes

This means the organization should be able to recover data to approximately five minutes before the failure.

Achieving RTO and RPO

Requirement	Technique
Low RTO	Automated failover, standby infrastructure
Low RPO	Frequent backups and continuous replication
Accidental deletion	Versioning
Regional failure	Cross-region replication
Database corruption	Point-in-time recovery

Business Continuity Steps

1. Identify critical applications and data.
2. Define RTO and RPO.

3. Enable automated backups.
4. Enable storage versioning.
5. Replicate important data to another region.
6. Maintain a disaster recovery environment.
7. Automate failover and recovery procedures.
8. Regularly test backup restoration.
9. Monitor backup and replication status.
10. Document the disaster recovery procedure.

Conclusion

A combination of automated backup, versioning, replication, point-in-time recovery, and failover provides strong protection against accidental deletion and infrastructure failures while meeting defined RTO and RPO requirements.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.

Some applications must remain on-premises due to compliance requirements.

Design a hybrid architecture that ensures secure communication between environments.

Explain how identity management and encryption should be implemented.

Discuss challenges in managing hybrid cloud environments.

Problem Statement

A government agency wants to use a **hybrid cloud** because some sensitive data must remain on-premises due to compliance requirements, while non-sensitive applications can use public cloud services.

Proposed Architecture

Government Users



Identity & Access Management



Secure Gateway / VPN / Private Connection



On-Premises Datacenter Public Cloud

Sensitive Data Non-Sensitive Applications

Government Databases Web Applications

Critical Systems Analytics / Public Services

1. On-Premises Environment

Sensitive information and applications subject to strict compliance requirements remain within the agency's own datacenter.

Examples:

- Confidential government records.
- Highly sensitive databases.
- Critical internal systems.

2. Public Cloud Environment

Non-sensitive workloads can be hosted in the public cloud.

Examples:

- Public websites.
- General-purpose applications.
- Public information portals.
- Scalable analytics workloads.

3. Secure Communication

Communication between on-premises infrastructure and the cloud should use:

- Site-to-site VPN.
- Dedicated private connectivity where required.
- Firewalls.
- Network segmentation.
- Intrusion detection/prevention.
- Secure routing.

4. Identity Management

A centralized identity management system should provide:

- Single Sign-On.
- Role-Based Access Control.
- Multi-Factor Authentication.
- Least-privilege access.
- Centralized identity lifecycle management.

Example:

Government Employee

↓

Identity Provider

↓

Authentication + MFA



Cloud / On-Premises Resources

5. Encryption

Data in transit: Use secure protocols such as TLS and encrypted VPN connections.

Data at rest: Encrypt databases, storage volumes, and backup data.

Encryption keys should be securely managed using a cloud key management service or dedicated key-management infrastructure.

Challenges

1. Complex management across two environments.
2. Different security policies.
3. Network latency between cloud and on-premises systems.
4. Identity integration challenges.
5. Compliance and auditing requirements.
6. Higher operational complexity.
7. Data synchronization difficulties.

Conclusion

A hybrid cloud architecture allows the government agency to keep sensitive workloads on-premises while using the scalability of public

cloud services for non-sensitive workloads. Secure networking, centralized identity management, encryption, and strict access controls are essential.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.

The system must continue operating even if one region goes down.

Propose a multi-region deployment strategy in the cloud.

Explain how data replication and failover mechanisms should be configured.

Discuss monitoring and alerting strategies for proactive issue detection.

Problem Statement

A SaaS provider requires its application to remain available even if an entire cloud region becomes unavailable. A multi-region architecture should therefore be implemented.

Proposed Architecture

Users



Global DNS / Traffic Manager



Region 1 Region 2

Load Balancer Load Balancer

↓ ↓

Application Servers Application Servers

↓ ↓

Database Database

↘ ↙

Data Replication

1. Multi-Region Deployment

The application should be deployed in at least two geographically separate cloud regions.

For example:

Primary Region → Active

Secondary Region → Standby/Active

If Region 1 fails, traffic can be redirected to Region 2.

2. Load Balancing

A global traffic management system distributes users across healthy regions.

It can monitor application health and stop sending traffic to an unhealthy region.

3. Data Replication

Application data should be replicated between regions.

Depending on business requirements, replication can be:

- Synchronous – Stronger consistency but potentially higher latency.
- Asynchronous – Lower latency but may result in some recent data loss during a sudden failure.

4. Failover Mechanism

Normal operation:

Users → Region 1

If Region 1 fails:

Users → Region 2

Health checks continuously monitor the availability of each region.

5. Database Failover

The database architecture should maintain a replicated database in the secondary region.

During failure:

1. Detect primary-region failure.
2. Promote the secondary database if required.
3. Redirect application traffic.

4. Verify application health.

5. Continue serving users from the healthy region.

6. Monitoring and Alerting

The SaaS provider should continuously monitor:

- CPU utilization.
- Memory.
- Application response time.
- Error rates.
- Database health.
- Network availability.
- Request volume.
- Replication lag.
- Availability of each region.

Alerts should be triggered when predefined thresholds are exceeded.

Example:

High Error Rate

↓

Monitoring System

↓

Alert

↓

Operations Team

↓

Failover / Corrective Action

Proactive Detection

Health checks and automated monitoring can detect problems before they cause a complete outage.

For example:

Increasing latency → Alert → Investigate

Region failure → Automatic failover → Secondary region serves users

Conclusion

A multi-region architecture with global load balancing, cross-region data replication, automated failover, health checks, and continuous monitoring provides high availability and fault tolerance. It allows the SaaS application to continue operating even if one entire cloud region becomes unavailable.

